
Q-Interference: Memory-Efficient Phase-Aware Quantum-Inspired Attention

Emama Nahid
Kennesaw State University
Marietta, GA, USA
enahid@students.kennesaw.edu

Tahmid Imtiaz Imu
Kennesaw State University
Marietta, GA, USA
timu1@students.kennesaw.edu

Huayue Gu
Kennesaw State University
Marietta, GA, USA
hgu2@kennesaw.edu

Liran Ma
Miami University
Oxford, OH, USA
mal18@MiamiOH.edu

Zhipeng Cai
Georgia State University
Atlanta, GA, USA
zcaigsu.edu

Honghui Xu
Kennesaw State University
Marietta, GA, USA
hxu10@kennesaw.edu

Abstract

GPT attention measures token compatibility through dot-product similarity. This mechanism is simple, effective, and memory-efficient. But it does not explicitly model whether strong token features should reinforce or suppress one another. We introduce Q-Interference, a fully classical quantum-inspired attention mechanism for autoregressive language modeling that augments each query and key feature with an amplitude and a learned phase. The resulting attention score is phase-aware which aligned phases contribute constructively while conflicting phases contribute destructively. Although Q-Interference yields a richer interaction rule than similarity alone, a naive implementation of Q-Interference requires a large token-pair-feature interaction tensor, making it memory-intensive and often impractical. To address this limitation, we propose an exact trigonometric factorization that computes the same score using two standard matrix multiplications avoiding materialization of the large intermediate tensor. Q-Interference fits directly into a Transformer block in GPT and leaves the remainder of the model architecture and next-token prediction objective unchanged. Experiments on public benchmark datasets and baseline models show that the proposed reformulation trains stably in a controlled GPT-style setting and provides a consistent memory advantage over naive phase-aware interference attention. These results support the specific contribution of this work: an exact memory-efficient reformulation that makes phase-aware interference attention practical within a standard GPT pipeline. Our code is available at <https://anonymous.4open.science/r/Q-Interference-Memory-Efficient-Quantum-Inspired-Attention-BDF9>.

1 Introduction

Transformer based GPT models have become a foundation of modern language processing because they can represent context across token sequences Vaswani et al. [2017], Devlin et al. [2019]. A token

is a basic text unit. Self attention is the mechanism that lets each token compare itself with other tokens in the same sequence. GPT style autoregressive models use causal self attention where each position attends only to previous positions to predict the next token Brown et al. [2020]. However, standard attention usually measures token compatibility through a dot product which mainly reflects feature similarity. This view may miss cases where strong features should support or suppress each other depending on context.

This challenge becomes more important in long context language modeling Dai et al. [2019]. Practical systems such as document question answering, scientific text modeling and retrieval augmented generation often depend on information spread across many tokens Beltagy et al. [2020], Lewis et al. [2020]. These settings require a model to connect nearby words with distant evidence. Standard dense attention lets each token compare with many earlier tokens. So its memory cost grows with sequence length Dao et al. [2022], Beltagy et al. [2020]. During autoregressive inference, the key value cache stores past keys and values and also grows as more tokens are generated Kang et al. [2026], Kwon et al. [2023]. Consequently, memory efficiency is not only a systems concern but also a condition for making richer attention rules practical at scale.

Prior research has improved attention and language model efficiency through better memory use, cheaper sequence computation and richer interaction structure Tay et al. [2022]. On the efficiency side, standard attention has been accelerated or approximated through memory aware exact computation, low rank structure, random feature estimation, structural routing and sparse access patterns Dao et al. [2022], Wang et al. [2020], Choromanski et al. [2020], Su et al. [2024], Zaheer et al. [2020]. These advances make long sequence modeling more practical, but they largely keep token compatibility tied to the standard query key similarity view. On the modeling side, quantum and quantum inspired attention has introduced richer ways to represent token relationships through state based interaction, circuit motivated computation and quantum style similarity Li et al. [2024], Chen et al. [2025], Kong et al. [2025], Kuznetsov et al. [2026]. Quantum inspired efficiency has also been used to reduce fine tuning cost and key value cache storage, but these contributions operate outside the internal score computation that creates the phase aware interaction tensor Chen et al. [2024], Kang et al. [2026]. Together, these studies suggest that attention can be made both more efficient and more expressive. However, they do not address the specific memory overhead created when phase aware pairwise feature interactions are implemented directly inside GPT style autoregressive language modeling. This leaves a gap for an exact tensor memory-efficient reformulation that preserves constructive and destructive phase interactions while avoiding the large intermediate interaction tensor.

We introduce Q-Interference to address this gap. It is a fully classical quantum inspired attention mechanism for autoregressive language modeling. The method augments each query and key feature with an amplitude and a learned phase. The amplitude controls the strength of a feature, while the phase controls how that feature interacts with another token. Aligned phases create constructive interaction, whereas conflicting phases create destructive interaction. A direct implementation of this idea requires a large token pair feature tensor. So we derive an exact factorization that computes the same score using two standard matrix multiplications. This makes the phase-aware computation memory efficient with respect to its additional interaction cost, while preserving the GPT backbone and the next token prediction objective. Our contributions are as follows.

- We introduce Q-Interference, a phase aware quantum inspired attention mechanism for GPT style language modeling. It uses amplitude and learned phase to model constructive and destructive token interactions.
- An exact memory efficient reformulation is derived for phase aware interference attention. The reformulation computes the same attention score with two standard matrix multiplications.
- Q-Interference is evaluated in a controlled GPT style setup. The backbone and next token objective remain unchanged so that the effect of the attention rule can be isolated.

2 Q-Interference

We introduce Q-Interference, a quantum-inspired variant of GPT in which the standard dot-product attention score is replaced by a phase-aware interference score. The model remains entirely classical and is trained on standard GPU hardware. The aim is not to claim quantum advantage, but to incorporate a useful structural idea from wave-like interactions into autoregressive language modeling. Notation and proof assumptions are collected in Appendix B.1. The design is intentionally minimal.

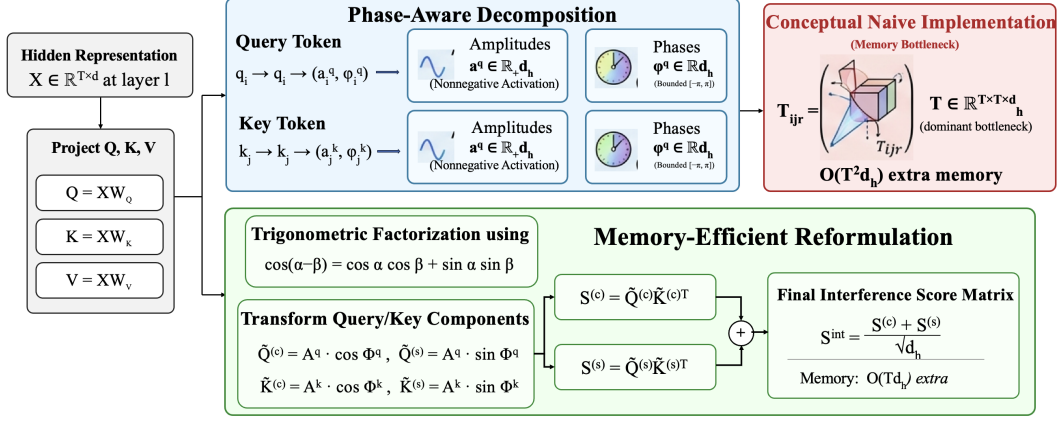


Figure 1: Q-Interference methodology flowchart.

We preserve the standard GPT backbone, including token and positional embeddings, residual connections, layer normalization, feed-forward blocks, and the next-token prediction objective. As shown in Fig. 1, the only modification lies in the attention scoring rule. This keeps the comparison against a standard GPT baseline controlled and makes the source of improvement easier to interpret. Let $X \in \mathbb{R}^{T \times d}$ denote the hidden sequence representation at a given layer, where T is the sequence length and d is the model dimension. Standard self-attention computes the query, key, and value projections as $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$ are learned projections for a head of dimension d_h . The conventional attention score between tokens i and j is $s_{ij}^{\text{std}} = \frac{q_i^\top k_j}{\sqrt{d_h}}$. After applying the causal mask M , the normalized attention weights are

$$\alpha_{ij} = \frac{\exp(s_{ij}^{\text{std}} + M_{ij})}{\sum_{m \leq i} \exp(s_{im}^{\text{std}} + M_{im})} \quad (1)$$

and the output at position i is $o_i = \sum_{j \leq i} \alpha_{ij} v_j$.

This reformulation is effective, but it assumes that token compatibility is sufficiently captured by magnitude-based similarity. Our method relaxes that assumption by introducing an explicit phase component into token interaction.

Phase-Aware Interference Attention: Standard self-attention measures token compatibility through the query-key dot product. Although effective, this score is mainly magnitude-driven and treats strong feature alignment as supportive interaction. In language, however, token relationships can be either reinforcing or suppressive depending on context. To model this more explicitly, we introduce a quantum-inspired attention mechanism that combines feature strength with relative phase alignment.

Our goal is not to claim quantum advantage, but to use a simple wave-inspired principle: aligned signals interact constructively, while misaligned signals interact destructively. This gives the attention score a more expressive way to capture both supportive and conflicting token relationships.

Instead of using real-valued query and key vectors alone, we represent each projected feature using an amplitude-phase decomposition:

$$q_i \rightarrow (a_i^q, \phi_i^q), k_j \rightarrow (a_j^k, \phi_j^k), \quad (2)$$

where $a_i^q, a_j^k \in \mathbb{R}_+^{d_h}$ are nonnegative amplitudes and $\phi_i^q, \phi_j^k \in \mathbb{R}^{d_h}$ are learned phases. In practice, amplitudes are produced through a nonnegative activation, while phases are constrained to a bounded interval such as $[-\pi, \pi]$ for numerical stability. This decomposition gives a simple interpretation. The amplitude controls how strongly a feature participates in attention, while the phase controls how that feature interacts with another one. Two features with aligned phase reinforce each other, whereas two features with conflicting phase suppress one another. In this way, token interaction is no longer determined only by feature strength, but also by relative phase alignment. Given a query token i and

a key token j , we define the interference-based attention score as

$$s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k) \quad (3)$$

This is the main departure from standard attention. Each feature contributes not only according to its amplitude, but also according to the cosine of its phase difference. When the phases are close, the cosine term is positive and the interaction is strengthened. When the phases disagree, the cosine term decreases or becomes negative, suppressing the interaction. As a result, the model can distinguish constructive and destructive relationships between tokens rather than treating all large-magnitude alignments as equally supportive. An algebraic derivation of the phase-aware interference score is provided in Appendix B.2.

The trade-off is that this richer interaction rule is more expensive than classical dot-product attention when implemented naively. In particular, directly computing phase-aware pairwise interactions introduces an additional intermediate structure over token pairs and feature dimensions, which significantly increases memory usage. This makes the method more expressive, but also potentially less practical at scale. To address this issue, we later introduce a memory-efficient factorization that preserves the same interference score while avoiding the explicit construction of the large intermediate tensor. Once the score is computed, the rest of the attention pipeline remains unchanged:

$\alpha_{ij} = \frac{\exp(s_{ij}^{\text{int}} + M_{ij})}{\sum_{m \leq i} \exp(s_{im}^{\text{int}} + M_{im})}$, $o_i = \sum_{j \leq i} \alpha_{ij} v_j$. Thus, the proposal changes only the compatibility function inside self-attention, while preserving the standard GPT transformer workflow.

Memory-Efficient Reformulation: The phase-aware attention defined above gives a richer compatibility score than standard dot-product attention by allowing token interactions to be reinforced or weakened through relative phase alignment. The drawback is that a direct implementation requires pairwise phase interactions across all token pairs and feature dimensions, which introduces a large intermediate tensor and high memory cost. Our method addresses this issue by retaining the phase-aware score while avoiding the main memory bottleneck of the naive computation. A direct implementation of the interference score is computationally impractical. The naive formulation constructs an interaction tensor over token pairs and feature dimensions: $\mathcal{T}_{ijr} = a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k)$. The final score is then obtained by summing over the feature index r : $s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} \mathcal{T}_{ijr}$.

However, $\mathcal{T} \in \mathbb{R}^{T \times T \times d_h}$ introduces a large intermediate tensor that quickly becomes the dominant memory bottleneck. This is exactly the inefficiency we aim to remove. Our key technical contribution is an exact trigonometric factorization of the interference score. Using the identity: $\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta$, we rewrite the score as

$$s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} (a_{i,r}^q \cos \phi_{i,r}^q) (a_{j,r}^k \cos \phi_{j,r}^k) + \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} (a_{i,r}^q \sin \phi_{i,r}^q) (a_{j,r}^k \sin \phi_{j,r}^k). \quad (4)$$

Then, we can define transformed query and key components:

$$\begin{aligned} \tilde{q}_{i,r}^{(c)} &= a_{i,r}^q \cos \phi_{i,r}^q, & \tilde{q}_{i,r}^{(s)} &= a_{i,r}^q \sin \phi_{i,r}^q, \\ \tilde{k}_{j,r}^{(c)} &= a_{j,r}^k \cos \phi_{j,r}^k, & \tilde{k}_{j,r}^{(s)} &= a_{j,r}^k \sin \phi_{j,r}^k. \end{aligned}$$

Substituting these definitions yields $s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} (\tilde{q}_i^{(c)\top} \tilde{k}_j^{(c)} + \tilde{q}_i^{(s)\top} \tilde{k}_j^{(s)})$. Finally, the full score matrix becomes

$$S^{\text{int}} = \frac{\tilde{Q}^{(c)} \tilde{K}^{(c)\top} + \tilde{Q}^{(s)} \tilde{K}^{(s)\top}}{\sqrt{d_h}}. \quad (5)$$

This reformulation is exact and introduces no approximation. It avoids explicitly forming the $T \times T \times d_h$ interaction tensor by rewriting the phase-aware score as two standard matrix multiplications and an addition. Consequently, the extra memory cost drops from $\mathcal{O}(T^2 d_h)$ in the naive form to $\mathcal{O}(T d_h)$ in the factorized form. The final score matrix remains standard attention-sized, so our claim is not to eliminate all quadratic costs of dense attention, but to remove the additional memory overhead introduced by naive phase-aware interaction. Formal proofs of exact score equivalence and phase-specific memory accounting are provided in Appendices B.3 and B.4.

Integration and Training: The proposed score is inserted into a standard GPT block without changing the rest of the model architecture. Let $H^{(\ell)}$ denote the hidden representation at layer ℓ . The layer update follows the usual residual form:

$$\hat{H}^{(\ell)} = H^{(\ell)} + \text{MHA}_{\text{int}}\left(\text{LN}\left(H^{(\ell)}\right)\right), \quad (6)$$

$$H^{(\ell+1)} = \hat{H}^{(\ell)} + \text{MLP}\left(\text{LN}\left(\hat{H}^{(\ell)}\right)\right). \quad (7)$$

Here, MHA_{int} denotes multi-head attention using the proposed interference score. All remaining components are inherited directly from the baseline GPT architecture. This keeps the architectural intervention narrow and makes the comparison with the original model more reliable. A proof of compatibility with the standard causal GPT attention interface is provided in Appendix B.5.

Training is performed with the standard autoregressive language modeling objective. Given a token sequence $(x_1, x_2, \dots, x_T)$, we minimize

$$\mathcal{L}_{\text{LM}} = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t}). \quad (8)$$

No auxiliary objective is introduced. This keeps the setup identical to standard autoregressive GPT training and isolates the effect of the proposed attention mechanism.

3 Experiment and Evaluation

We evaluate Q-Interference in a controlled GPT-style autoregressive language modeling setting, where the standard scaled dot-product attention score is replaced by the proposed phase-aware interference score while the rest of the GPT transformer backbone is kept unchanged. This design keeps token embeddings, positional embeddings, residual connections, layer normalization, feed-forward blocks, and the next-token prediction objective fixed, so that observed differences can be attributed primarily to the attention mechanism itself. Our experiments are designed to answer three questions. **RQ1:** whether Q-Interference can be trained successfully inside a standard GPT pipeline? **RQ2:** whether the exact factorization removes the extra memory overhead introduced by naive phase-aware interaction? **RQ3:** how the resulting model compares with standard and pre-trained GPT transformer baselines in terms of language-modeling quality and efficiency?

3.1 Experiment Setup

Dataset and Preprocessing: We use four language modeling datasets: WikiText-103 Merity et al. [2017], TinyStories Li and Eldan [2024], pile-10k Gao et al. [2020], and small-C4 Raffel et al. [2020]. WikiText-103 serves as the main controlled benchmark, while the other three datasets are used to evaluate cross-dataset behavior under different data regimes. We use the GPT-2 tokenizer and segment each corpus into fixed-length sequences with context length 512.

Baselines: We compare six models. We first consider two baselines: a standard GPT baseline with conventional causal self-attention and a Q-GPT: quantum inspired baseline adopted from a previously published quantum-inspired GPT study Liao and Ferrie [2024], which serves as a reference model architectural comparison and is not our contribution. We then include two widely used pre-trained decoder-only reference models, GPT-Neo-125M Black et al. [2021] and OPT-125M Zhang et al. [2022]. To clarify the evaluation design, Table 1 summarizes the role of each baseline and why the selected set is sufficient for the scope of this study.

Training Configurations: Finally, we evaluate two versions of our phase-aware model family: a naive interference model that directly computes the phase-aware interaction tensor, and the proposed Q-Interference model, which uses the exact memory-efficient reformulation. The naive model is used primarily for profiling and ablation because it is substantially more memory-intensive. For the main controlled comparison, the standard GPT baseline has approximately 124.0M parameters, while the matched Q-Interference model has approximately 123.7M parameters, using 12 layers, 12 heads, and model dimension 720. Unless otherwise stated, all models are trained with the standard autoregressive objective on NVIDIA Tesla V100-SXM2-32GB GPUs using mixed precision.

Table 1: Baseline selection rationale.

Model	Comparison role	Justification
Baseline GPT	Standard internal control	Directly controlled reference for conventional causal dot-product attention under the same GPT-style training setup.
Q-GPT baseline	Quantum-inspired reference	Provides a published quantum-inspired GPT-style architectural comparison in this setting.
OPT-125M and GPT-Neo-125M	External pretrained reference	Gives context against an open autoregressive GPT-style model family.

Evaluation Metrics: We report validation loss, test loss, and test perplexity as the primary language-modeling metrics. To evaluate practicality, we also report peak GPU memory usage, elapsed execution time, and sequence-length scaling behavior. Peak GPU memory was measured under fixed batch size, context length, numerical precision, and hardware; therefore, the reported peak memory values mainly reflect the computational behavior of the model architecture rather than the identity of the dataset itself. This is why peak memory can remain nearly constant across datasets for a given model. Our goal is not to eliminate the standard quadratic attention matrix, but to remove the additional memory cost introduced by naive phase-aware interaction while preserving the same interference-based scoring idea. Definitions of the reported metrics and model categories are summarized in Appendix C.1.

3.2 Comparison Evaluation with Baselines

In this section, Q-Interference is evaluated from two comparison perspectives. First, the proposed model is compared with the baselines across the evaluated datasets and it is compared against 125M pre-trained reference models.

Comparison across datasets: We evaluate the standard GPT baseline, the proposed Q-Interference model, and the Q-GPT baseline across four datasets: WikiText-103, TinyStories, pile-10k, and small-C4. WikiText-103 serves as the main benchmark, while the remaining datasets are used to examine whether the observed behavior transfers across different language-modeling regimes. In all cases, the comparison is performed under the same overall GPT-style training pipeline so that the effect of the proposed phase-aware hybrid attention can be isolated while keeping the backbone architecture and next-token prediction objective unchanged. Table 2 shows a consistent practical advantage for Q-Interference together with mixed but competitive quality behavior. On the main benchmark, WikiText-103, Q-Interference is the strongest internal model, improving validation loss, test loss, and test perplexity over the standard GPT baseline while also reducing peak training GPU memory from 8055.76 MB to 4227.14 MB. A visual summary of the WikiText-103 comparison is provided in Appendix C.2. On TinyStories, it remains very close to the standard GPT baseline in test loss and perplexity while preserving the same memory advantage. On pile-10k and small-C4, the standard GPT baseline remains stronger in final test quality, but Q-Interference is still substantially more memory-efficient and remains clearly stronger than the Q-GPT baseline. Figure 2 shows the relationship between test perplexity and peak GPU memory across datasets, highlighting that Q-Interference achieves the most favorable practical trade-off within the phase-aware model family. Overall, these results suggest that the proposed method is best understood not as a universal replacement for standard GPT, but as the strongest practical model within the phase-aware family, offering the most favorable quality-efficiency trade-off across the evaluated datasets. A supplementary visualization of memory behavior under the fixed profiling setup is provided in Appendix C.3.

Comparison with pre-trained reference models: To further contextualize our results, we compare Q-Interference with two widely used pretrained decoder-only reference models, GPT-Neo-125M and OPT-125M. Since these are not parameter-matched internal baselines, we treat them as pre-trained reference points rather than direct apples-to-apples competitors. As shown in Table 3, GPT-Neo-125M and OPT-125M achieve stronger final language-modeling quality than Q-Interference on all four datasets, with the gap being especially large on pile-10k and small-C4. At the same time, Q-Interference remains more memory-efficient on TinyStories, pile-10k, and small-C4, and is competitive with these models on WikiText-103 in terms of training memory. Thus, the results position Q-Interference not as a universal replacement for strong pretrained GPT transformers, but as the strongest practical phase-aware model in our experiments with a favorable efficiency profile. A supplementary visual comparison to the reference models is provided in Appendix C.4.

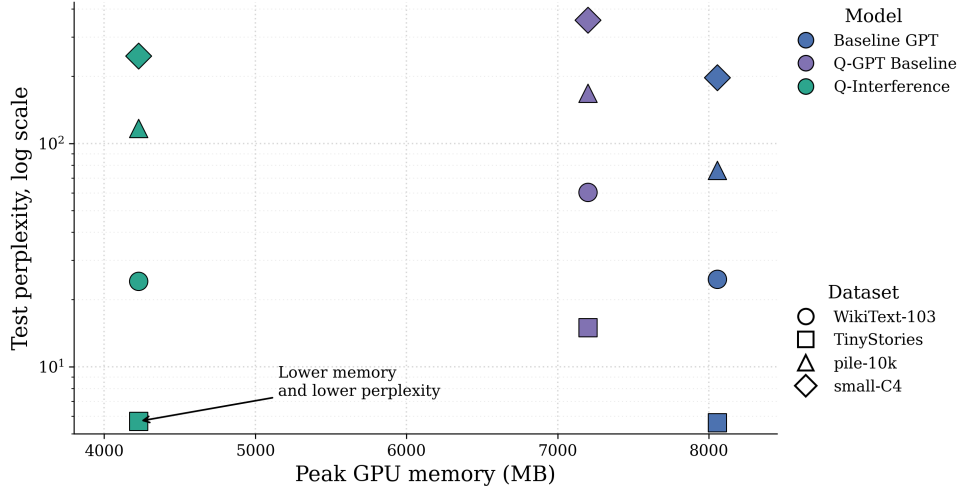


Figure 2: Quality-memory trade-off for the main model comparison.

Table 2: Baselines vs Q-Interference comparison over benchmark dataset

Dataset	Model	Best Val Loss	Test Loss	Test PPL	Peak GPU Mem (MB)
WikiText-103	Baseline GPT	3.2036	3.2049	24.6534	8055.76
WikiText-103	Q-GPT baseline	4.1142	4.1032	60.5367	7199.26
WikiText-103	Q-Interference (ours)	3.1809	3.1852	24.1718	4227.14
TinyStories	Baseline GPT	1.7279	1.7263	5.6196	8055.76
TinyStories	Q-GPT baseline	2.7036	2.7082	15.0016	7199.26
TinyStories	Q-Interference (ours)	1.7344	1.7394	5.6941	4227.14
pile-10k	Baseline GPT	4.1993	4.3294	75.9008	8055.76
pile-10k	Q-GPT baseline	4.9723	5.1241	168.0210	7199.26
pile-10k	Q-Interference (ours)	4.0745	4.7616	116.9275	4227.14
small-C4	Baseline GPT	5.3191	5.2833	197.0284	8055.76
small-C4	Q-GPT baseline	5.9414	5.8769	356.7006	7199.26
small-C4	Q-Interference (ours)	5.5224	5.5090	246.9012	4227.14

Overall, the results show that Q-Interference is the strongest practical model within the proposed phase-aware family. It achieves the best internal matched result on WikiText-103 and offers the most favorable quality-memory trade-off across the custom models we study. While the gains are not universal across all datasets, the proposed design consistently remains more practical than the Q-GPT baseline.

3.3 Ablation Study

We design the ablation study to isolate the two main claims of this work. First, we test whether the memory-efficient reformulation is necessary for making phase-aware attention practical. Second, we test whether the phase component itself contributes to language-modeling quality inside the final hybrid design. Unlike the main results section, which emphasizes overall model comparison, the goal here is to separate the systems contribution from the modeling contribution.

Naive phase-aware attention versus memory-reduced Q-Interference: The first ablation compares the naive phase-aware implementation directly with the final Q-Interference model across the evaluated datasets. The naive model uses the same interference-based scoring idea, but computes it without the memory-efficient reformulation, and therefore serves as the most direct control for testing whether the proposed reformulation is necessary in practice. As shown in Table 4, the systems motivation of the paper is directly supported. On WikiText-103, the full matched naive run is impractical and reaches out-of-memory at context length 512, whereas Q-Interference remains fully trainable. On TinyStories, pile-10k, and small-C4, peak training memory is reduced from 12138.34 MB to 4227.14 MB, corresponding to a reduction of about 65%. At the same time, stronger final test quality is achieved on the completed datasets. Although the naive model attains a slightly lower validation loss

Table 3: Comparison to pre-trained reference models.

Dataset	Model	Test Loss	Test PPL	Peak GPU Mem (MB)
WikiText-103	GPT-Neo-125M	2.9673	19.4386	4097.91
WikiText-103	OPT-125M	3.0169	20.4278	4213.64
WikiText-103	Q-Interference (ours)	3.1852	24.1718	4227.14
TinyStories	GPT-Neo-125M	1.5198	4.5713	9934.54
TinyStories	OPT-125M	1.4217	4.1443	6219.52
TinyStories	Q-Interference (ours)	1.7394	5.6941	4227.14
pile-10k	GPT-Neo-125M	2.3898	10.9116	9934.54
pile-10k	OPT-125M	2.7873	16.2376	6219.52
pile-10k	Q-Interference (ours)	4.7616	116.9275	4227.14
small-C4	GPT-Neo-125M	3.5453	34.6512	9931.54
small-C4	OPT-125M	3.4794	32.4405	6219.52
small-C4	Q-Interference (ours)	5.5090	246.9012	4227.14

Table 4: Naive phase-aware attention versus Q-Interference.

Dataset	Model	Best Val Loss	Test Loss	Test PPL	Peak GPU Mem (MB)
WikiText-103	Naive interference	OOM	OOM	OOM	OOM
WikiText-103	Q-Interference (ours)	3.1809	3.1852	24.1718	4227.14
TinyStories	Naive interference	2.2972	2.3422	10.4037	12138.34
TinyStories	Q-Interference (ours)	1.7344	1.7394	5.6941	4227.14
pile-10k	Naive interference	4.0223	5.1086	165.4355	12138.34
pile-10k	Q-Interference (ours)	4.0745	4.7616	116.9275	4227.14
small-C4	Naive interference	5.8665	5.8401	268.3050	12138.34
small-C4	Q-Interference (ours)	5.5224	5.5090	246.9012	4227.14

on pile-10k, it performs worse in final test loss and perplexity. Overall, these results indicate that the efficient reformulation is not a minor implementation detail but the key step that makes phase-aware attention practical.

Effect of the phase component: The second ablation is designed to evaluate whether the phase term contributes meaningfully within the final hybrid design. For this purpose, the full Q-Interference model is compared with a phase-disabled control in which the same hybrid architecture and training setup are retained, but the learned phase contribution is removed. The corresponding results are reported in Table 5. Overall, a small but consistent improvement in final test quality is observed when the phase term is retained. Across all four datasets, slightly better test loss and perplexity are achieved by the full model than by the phase-disabled variant, while only a modest difference in memory is observed. These results suggest that the main practical benefit of the method is provided by the memory-efficient reformulation, whereas an additional but smaller modeling gain is supplied by the phase term within the final hybrid design.

Taken together, a two-part conclusion is supported by the ablations. First, the exact memory-efficient reformulation is shown to be necessary for practicality, since naive phase-aware attention is either highly memory-intensive or impractical under the full matched setting. Second, a consistent but modest improvement in final test quality is provided by the phase term across datasets. Thus, the primary systems contribution of this work is established as the step that makes phase-aware attention usable in practice, while an additional modeling benefit is contributed by the phase component within the final hybrid design.

4 Related Work

Quantum and quantum-inspired attention for language modeling Recent work has begun to explore whether ideas from quantum computation can enrich GPT-style attention. Early studies such as Li et al. [2024] and Chen et al. [2025] introduced quantum self-attention mechanisms for NLP by representing token interactions through quantum-inspired or quantum-state-based formulations. These works showed that quantum-style attention can improve representation quality, but they were evaluated mainly on text classification tasks rather than autoregressive language modeling. As a result, they do not directly address the requirements of GPT-style next-token prediction, where attention must operate efficiently over long causal contexts. More recent work moves closer to generative GPT

Table 5: Phase ablation for the final Q-Interference model.

Dataset	Model	Best Val Loss	Test Loss	Test PPL	Peak GPU Mem (MB)
WikiText-103	Q-Interference (ours)	3.1809	3.1852	24.1718	4227.14
WikiText-103	Q-Interference w/o phase	3.1830	3.1879	24.2373	3993.53
TinyStories	Q-Interference (ours)	1.7344	1.7394	5.6941	4227.14
TinyStories	Q-Interference w/o phase	1.7352	1.7410	5.7030	3993.53
pile-10k	Q-Interference (ours)	4.0745	4.7616	116.9275	4227.14
pile-10k	Q-Interference w/o phase	4.0736	4.7634	117.1429	3993.53
small-C4	Q-Interference (ours)	5.5224	5.5090	246.9012	4227.14
small-C4	Q-Interference w/o phase	5.5228	5.5093	246.9672	3993.53

transformer settings. Smaldone et al. [2025] introduces a hybrid quantum-classical self-attention module in a decoder for sequence generation, and Kong et al. [2025] extends hybrid quantum-classical transformers to natural language generation. Most relevant to our setting, Kuznetsov et al. [2026] incorporates a classical quantum-inspired self-attention mechanism into the autoregressive GPT-1 pipeline. Together, these studies demonstrate that quantum or quantum-inspired attention can be used in generative transformer models. However, they mainly emphasize feasibility and architectural design, rather than the memory overhead introduced by richer attention interactions. This limitation is especially important for quantum-inspired attention designs that introduce additional structure beyond standard dot-product similarity. Although such mechanisms can model richer token relationships, prior work has not directly addressed the specific memory overhead caused by explicitly computing phase-aware pairwise interactions. Our work focuses on this issue by reformulating the computation to avoid constructing the large intermediate interaction tensor.

Memory optimization in quantum-inspired LLM research A separate line of research uses quantum or quantum-inspired ideas for memory-efficient adaptation of large language models. Chen et al. [2024], Liu et al. [2025], and Raj and Coyle [2026] all aim to reduce the cost of fine-tuning by introducing parameter-efficient adaptation schemes motivated by quantum circuits or quantum parameter generation. These methods are valuable for reducing the trainable footprint of LLM adaptation, but they operate at the level of fine-tuning parameters or adapters, not at the level of attention-score computation itself. Therefore, they do not address the memory cost that arises when a richer attention rule introduces an explicit token-pair-feature interaction tensor.

More recently, Kang et al. [2026] studies memory reduction during inference by compressing the KV cache with a quantum-inspired probabilistic representation. This is closely related to the broader goal of efficient long-context language modeling, but it addresses a different source of memory cost. Specifically, KV-cache compression reduces the storage required for past keys and values during decoding, whereas our method targets the additional intermediate memory introduced by a naive implementation of phase-aware interference attention. In this sense, prior quantum-inspired memory-efficiency work has focused on cache compression, while our contribution lies in an exact reformulation that avoids explicitly constructing the internal interaction tensor.

Taken together, these prior studies show that quantum-inspired attention can enrich token interaction and that memory optimization has also been explored in adjacent parts of LLM systems. Our method is positioned differently: it begins from a more expressive phase-aware attention score and then derives an exact trigonometric factorization that avoids materializing the $T \times T \times d_h$ interaction tensor. We do not claim to remove the quadratic cost of dense attention altogether; rather, we specifically reduce the additional phase-specific memory overhead introduced by naive interference computation while preserving the same score exactly. To the best of our knowledge, such an exact memory-efficient factorization for phase-aware quantum-inspired attention in a GPT-style model has not been previously reported. A compact summary of representative prior work relative to Q-Interference is provided in the appendix A.

5 Conclusion

Q-Interference was introduced as a fully classical quantum-inspired attention mechanism for autoregressive language modeling, where token interactions are modeled through amplitude and learned phase. It is considered quantum-inspired because it draws on a wave-interference principle: aligned phases contribute constructively, while conflicting phases contribute destructively. This is important

because standard attention mainly measures similarity, whereas phase-aware interaction provides a richer way to capture both supportive and suppressive relationships between tokens. Since a naive implementation is highly memory-intensive, an exact trigonometric factorization was derived to make the same interaction practical within a standard GPT pipeline. Across controlled experiments on benchmark datasets, Q-Interference was found to be the strongest practical model within the proposed phase-aware family, providing a favorable balance between language modeling quality and memory efficiency. The results further showed that the exact memory-efficient reformulation is essential for practicality, while the phase component contributes a small but consistent modeling benefit within the final hybrid design. Overall, these findings suggest that quantum-inspired phase interactions offer a meaningful extension of standard attention, but their usefulness depends on an exact reformulation that makes them practical at scale.

Acknowledgments and Disclosure of Funding

Use unnumbered first level headings for the acknowledgments. All acknowledgments go at the end of the paper before the list of references. Moreover, you are required to declare funding (financial activities supporting the submitted work) and competing interests (related financial activities outside the submitted work). More information about this disclosure can be found at: <https://neurips.cc/Conferences/2026/PaperInformation/FundingDisclosure>.

Do **not** include this section in the anonymized submission, only in the final paper. You can use the ack environment provided in the style file to automatically hide this section in the anonymized submission.

References

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019. URL <https://api.semanticscholar.org/CorpusID:52967399>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *ArXiv*, abs/1901.02860, 2019. URL <https://api.semanticscholar.org/CorpusID:57759363>.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *ArXiv*, abs/2004.05150, 2020. URL <https://api.semanticscholar.org/CorpusID:215737171>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, 2022.

- Jieui Kang, Jaeyoung Choi, Wonhui Noh, and Jaehyeong Sim. Qubitcache: Quantum-inspired probabilistic attention preservation for kv-cache compression. *IEEE Access*, 14:1–1, 2026. doi: 10.1109/ACCESS.2026.3680126.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Comput. Surv.*, 55(6), December 2022. ISSN 0360-0300. doi: 10.1145/3530811. URL <https://doi.org/10.1145/3530811>.
- Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *ArXiv*, abs/2006.04768, 2020. URL <https://api.semanticscholar.org/CorpusID:219530577>.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers, 2020.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomput.*, 568(C), February 2024. ISSN 0925-2312. doi: 10.1016/j.neucom.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: transformers for longer sequences. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Guangxi Li, Xuanqiang Zhao, and Xin Wang. Quantum self-attention neural networks for text classification. *Science China Information Sciences*, 67:1–13, 2024. doi: 10.1007/s11432-023-3879-7.
- Fu Chen, Qinglin Zhao, Li Feng, Chuangtao Chen, Yangbin Lin, and Jianhong Lin. Quantum mixed-state self-attention network. *Neural Networks*, 185:107123, 2025. doi: 10.1016/j.neunet.2025.107123.
- Desheng Kong, Xiangshuo Cui, Jiaying Jin, Jing Xu, and Donglin Wang. Hybrid quantum transformer for language generation, 2025. URL <https://arxiv.org/abs/2511.10653>.
- Nikita Kuznetsov, Niyaz Ismagilov, and Ernesto Campos. Quantum-inspired self-attention in a large language model, 2026. URL <https://arxiv.org/abs/2603.03318>.
- Zhuo Chen, Rumen Dangovski, Charlotte Loh, Owen Dugan, Di Luo, and Marin Soljačić. Quanta: Efficient high-rank fine-tuning of llms with quantum-informed tensor adaptation. In *Advances in Neural Information Processing Systems*, 2024.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Byj72udxe>.
- Yuanzhi Li and Ronen Eldan. Tinstories: How small can language models be and still speak coherent english, 2024. URL <https://openreview.net/forum?id=yiPtWSrBrN>.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 12 2020.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), January 2020. ISSN 1532-4435.

Yidong Liao and Chris Ferrie. Gpt on a quantum computer, 2024. URL <https://arxiv.org/abs/2403.09418>.

Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large scale autoregressive language modeling with meshtensorflow, October 2021.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models. *ArXiv*, abs/2205.01068, 2022. URL <https://api.semanticscholar.org/CorpusID:248496292>.

Anthony M. Smaldone, Yu Shee, Gregory W. Kyro, Marwa H. Farag, Zohim Chandani, Elica Kyoseva, and Victor S. Batista. A Hybrid Transformer Architecture with a Quantized Self-Attention Mechanism Applied to Molecular Generation. *Journal of Chemical Theory and Computation*, 21(10):5143–5154, May 2025. doi: 10.1021/acs.jctc.5c00331.

Chen-Yu Liu, Chao-Han Huck Yang, Hsi-Sheng Goan, and Min-Hsiu Hsieh. A quantum circuit-based compression perspective for parameter-efficient learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=bB00KNpznP>.

Snehal Raj and Brian Coyle. QuIC: Quantum-inspired compound adapters for parameter efficient fine-tuning, 2026. URL <https://openreview.net/forum?id=0J3tzHSIis>.

A Additional Related Work Summary

Table 6 provides a compact summary of representative prior work relative to Q-Interference. This table is intended as a supplementary overview of the related work discussion in the main paper, highlighting differences in GPT-style applicability, quantum-inspired design, phase-aware interaction, memory-awareness, and exact memory-efficient reformulation.

Table 6: Compact comparison of representative prior work and Q-Interference.

Work	GPT-style	Quantum-inspired	Phase-aware	Memory-aware	Exact memory-efficient
QSANN	×	✓	×	×	×
QMSAN	×	✓	×	×	×
HyQuT	✓	✓	×	×	×
QISA	✓	✓	×	×	×
QubitCache	✓	✓	×	✓	×
Q-Interference (ours)	✓	✓	✓	✓	✓

B Proofs and Derivations for Q-Interference

B.1 Notation and Assumptions

This appendix gives the algebraic details behind the Q-Interference attention reformulation. All derivations are written for a single attention head, since multi-head attention applies the same computation independently to each head. Let T denote the sequence length and let d_h denote the dimension of one attention head. For token i and token j , we denote the nonnegative query and key amplitude vectors by $a_i^q, a_j^k \in \mathbb{R}_+^{d_h}$, and the corresponding phase vectors by $\phi_i^q, \phi_j^k \in \mathbb{R}^{d_h}$. The proofs below concern exact algebraic equivalence and memory accounting, not generalization performance. The factorized computation still produces a standard $T \times T$ attention score matrix; it only avoids the additional $T \times T \times d_h$ token-pair-feature tensor used by the naive phase-aware implementation.

Table 7: Notation used in the proof appendix.

Symbol	Meaning
T	Sequence length.
d_h	Dimension of one attention head.
$a_i^q \in \mathbb{R}_+^{d_h}$	Nonnegative query amplitude vector for token i .
$a_j^k \in \mathbb{R}_+^{d_h}$	Nonnegative key amplitude vector for token j .
$\phi_i^q \in \mathbb{R}^{d_h}$	Query phase vector for token i .
$\phi_j^k \in \mathbb{R}^{d_h}$	Key phase vector for token j .
$\tilde{Q}^{(c)}, \tilde{K}^{(c)}$	Cosine-transformed query and key matrices.
$\tilde{Q}^{(s)}, \tilde{K}^{(s)}$	Sine-transformed query and key matrices.
S^{int}	Phase-aware interference attention score matrix.

B.2 Derivation of the Phase-Aware Interference Score

We first show where the phase-aware interference score comes from. For each feature dimension r , we represent the query and key features as complex amplitude-phase terms:

$$z_{i,r}^q = a_{i,r}^q e^{i\phi_{i,r}^q}, \quad z_{j,r}^k = a_{j,r}^k e^{i\phi_{j,r}^k}, \quad (9)$$

where i is the imaginary unit and $\overline{z_{j,r}^k}$ denotes the complex conjugate of $z_{j,r}^k$.

Proposition 1. *The phase-aware interference score is the scaled real part of an amplitude-phase inner product.*

Proof. For a query feature $z_{i,r}^q$ and a key feature $z_{j,r}^k$, their conjugate product is

$$z_{i,r}^q \overline{z_{j,r}^k} = a_{i,r}^q a_{j,r}^k e^{i(\phi_{i,r}^q - \phi_{j,r}^k)}. \quad (10)$$

Using Euler’s identity, the real part of this product is

$$\text{Re} \left(z_{i,r}^q \overline{z_{j,r}^k} \right) = a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k). \quad (11)$$

Summing this real-valued interaction over all feature dimensions gives

$$\text{Re} \left(\sum_{r=1}^{d_h} z_{i,r}^q \overline{z_{j,r}^k} \right) = \sum_{r=1}^{d_h} a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k). \quad (12)$$

After applying the standard attention scaling factor, the phase-aware interference score becomes

$$s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k). \quad (13)$$

Therefore, the proposed score is exactly the scaled real part of the amplitude-phase inner product. $\square$

This derivation also explains the constructive and destructive behavior of the score. When the phase difference $\phi_{i,r}^q - \phi_{j,r}^k$ is close to zero, the cosine term is positive and the feature interaction is reinforced. When the phase difference is large, the cosine term decreases and can become negative, which suppresses the feature interaction.

B.3 Exact Factorization of Phase-Aware Attention

We now show that the memory-efficient reformulation computes exactly the same score as the naive phase-aware interference score. The result follows from a direct trigonometric factorization and does not introduce any approximation.

Theorem 1. *For any query and key amplitude-phase representations, the factorized score matrix is exactly equal to the naive phase-aware interference score matrix.*

Proof. For a query token i , key token j , and feature dimension r , the naive phase-aware interaction is

$$a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k). \quad (14)$$

Using the identity

$$\cos(\alpha - \beta) = \cos \alpha \cos \beta + \sin \alpha \sin \beta, \quad (15)$$

we can rewrite this interaction as

$$a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k) = a_{i,r}^q a_{j,r}^k \cos \phi_{i,r}^q \cos \phi_{j,r}^k + a_{i,r}^q a_{j,r}^k \sin \phi_{i,r}^q \sin \phi_{j,r}^k. \quad (16)$$

Define the transformed query and key components as

$$\tilde{q}_{i,r}^{(c)} = a_{i,r}^q \cos \phi_{i,r}^q, \quad \tilde{q}_{i,r}^{(s)} = a_{i,r}^q \sin \phi_{i,r}^q, \quad (17)$$

and

$$\tilde{k}_{j,r}^{(c)} = a_{j,r}^k \cos \phi_{j,r}^k, \quad \tilde{k}_{j,r}^{(s)} = a_{j,r}^k \sin \phi_{j,r}^k. \quad (18)$$

Substituting these definitions into the interference score gives

$$s_{ij}^{\text{int}} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} \left(\tilde{q}_{i,r}^{(c)} \tilde{k}_{j,r}^{(c)} + \tilde{q}_{i,r}^{(s)} \tilde{k}_{j,r}^{(s)} \right). \quad (19)$$

This can be written as the sum of two inner products:

$$s_{ij}^{\text{int}} = \frac{\tilde{q}_i^{(c)\top} \tilde{k}_j^{(c)} + \tilde{q}_i^{(s)\top} \tilde{k}_j^{(s)}}{\sqrt{d_h}}. \quad (20)$$

Stacking all query and key vectors across the T tokens gives

$$S^{\text{int}} = \frac{\tilde{Q}^{(c)} \tilde{K}^{(c)\top} + \tilde{Q}^{(s)} \tilde{K}^{(s)\top}}{\sqrt{d_h}}. \quad (21)$$

Thus, every entry of the factorized score matrix equals the corresponding naive phase-aware interference score. The reformulation is therefore exact and introduces no approximation. $\square$

This factorization avoids explicitly constructing the additional token-pair-feature tensor required by the naive implementation. It does not remove the standard $T \times T$ attention score matrix.

B.4 Memory Cost of Naive and Factorized Computation

We now compare the additional phase-specific intermediate storage required by the naive and factorized implementations. This analysis excludes the standard dense attention score matrix $S^{\text{int}} \in \mathbb{R}^{T \times T}$, which is still produced in both cases.

Proposition 2. *The naive phase-aware implementation requires additional intermediate storage of order $T^2 d_h$, while the factorized implementation requires additional phase-specific storage of order $T d_h$, excluding the standard attention score matrix.*

Proof. In the naive implementation, the phase-aware interaction is materialized for every query token i , key token j , and feature dimension r . This gives the intermediate tensor

$$\mathcal{T}_{ijr} = a_{i,r}^q a_{j,r}^k \cos(\phi_{i,r}^q - \phi_{j,r}^k), \quad \mathcal{T} \in \mathbb{R}^{T \times T \times d_h}. \quad (22)$$

Therefore, the naive implementation requires storing $T^2 d_h$ additional interaction entries before summing over the feature dimension. Its additional phase-specific intermediate storage is therefore

$$\Theta(T^2 d_h). \quad (23)$$

In the factorized implementation, the computation uses the transformed matrices

$$\tilde{Q}^{(c)}, \tilde{Q}^{(s)}, \tilde{K}^{(c)}, \tilde{K}^{(s)} \in \mathbb{R}^{T \times d_h}. \quad (24)$$

These four matrices require $4Td_h$ entries in total. Ignoring constant factors, the additional phase-specific storage is therefore

$$\Theta(4Td_h) = \Theta(Td_h). \quad (25)$$

The factorized computation still forms the standard attention score matrix

$$S^{\text{int}} = \frac{\tilde{Q}^{(c)} \tilde{K}^{(c)\top} + \tilde{Q}^{(s)} \tilde{K}^{(s)\top}}{\sqrt{d_h}} \in \mathbb{R}^{T \times T}. \quad (26)$$

Thus, the factorization does not remove the standard dense attention matrix. It removes the additional $T \times T \times d_h$ token-pair-feature tensor required by the naive phase-aware implementation. $\square$

B.5 Compatibility with Causal GPT Attention

We finally show that Q-Interference preserves the causal self-attention interface used in GPT-style models. The proposed method changes only the score function and leaves masking, normalization, value aggregation, and the remaining GPT transformer block unchanged.

Lemma 1. *Replacing the standard dot-product score with S^{int} preserves the causal attention interface.*

Proof. For a sequence of length T , the factorized interference computation produces a score matrix

$$S^{\text{int}} \in \mathbb{R}^{T \times T}. \quad (27)$$

This has the same shape as the standard dot-product attention score matrix. Let $M \in \mathbb{R}^{T \times T}$ denote the causal mask, where entries above the causal boundary are assigned large negative values before softmax. Let $V \in \mathbb{R}^{T \times d_h}$ denote the value matrix for one attention head. The masked attention weights are computed as

$$A = \text{softmax}(S^{\text{int}} + M), \quad A \in \mathbb{R}^{T \times T}, \quad (28)$$

where the softmax is applied row-wise. The attention output is then

$$O = AV, \quad O \in \mathbb{R}^{T \times d_h}. \quad (29)$$

Thus, the output of the proposed attention head has the same shape as the output of a standard causal attention head. Therefore, residual connections, layer normalization, feed-forward blocks, and the next-token prediction objective can be used without modification. $\square$

C Additional Main Result Analysis

C.1 Metric and Model Definitions

Table 8 defines the main terms used in the appendix figures. All quality metrics are computed for autoregressive next-token prediction. Peak GPU memory is reported under the fixed training setup used in the main experiments. These definitions clarify how to read the quality and efficiency comparisons in the following subsections.

C.2 Controlled WikiText-103 Result

WikiText-103 is the main controlled dataset in our experiments. Figure 3 summarizes the internal comparison using two metric rulers. The top ruler reports test perplexity and the bottom ruler reports peak training GPU memory. Lower values are better for both metrics. Q-Interference obtains the lowest test perplexity among the internal models on WikiText-103. It also reduces peak GPU memory from 8055.76 MB for the standard GPT baseline to 4227.14 MB, which corresponds to an approximate 47.5% reduction while preserving the same GPT-style training objective.

Table 8: Definitions of metrics and model terms used in the appendix result analysis.

Term	Definition
Best validation loss	The lowest validation cross entropy observed during training. It is used to track model selection and training progress. Lower is better.
Test loss	The cross entropy loss measured on the held-out test split after training. It measures how well the model assigns probability to the correct next token. Lower is better.
Test perplexity	The exponential form of test loss. It can be read as the model’s effective uncertainty when predicting the next token. Lower is better.
Peak training GPU memory	The maximum GPU memory used during training under the reported batch size, context length, precision, and hardware. It measures practical training cost rather than language quality. Lower is better.
Memory reduction	The percentage decrease in peak GPU memory relative to a chosen baseline. A larger reduction means the model needs less GPU memory under the same reported setting.
Parameter matched comparison	A comparison where models have similar parameter counts. This helps reduce the chance that differences are caused only by model size.
Internal custom model	A model trained in our controlled experimental pipeline. This includes the standard GPT baseline, Q-GPT baseline, naive interference model, and Q-Interference.
Pre-trained reference model	A pretrained model used only for context. GPT-Neo-125M and OPT-125M are not treated as parameter matched internal baselines.
Naive interference model	A direct phase-aware implementation that materializes the token-pair-feature interaction tensor. It is used to show the memory cost without the exact efficiency.
Q-Interference	The proposed phase-aware model with exact memory-efficient factorization. It computes the same interference score while avoiding the large interaction tensor.

C.3 Cross-Dataset Memory Behavior

Figure 4 shows the peak training GPU memory of the custom model family on TinyStories, pile-10k, and small-C4. The naive interference model uses the largest memory because it directly materializes the phase-aware token-pair-feature interaction tensor. In contrast, Q-Interference avoids this tensor through the exact factorized computation. As a result, Q-Interference has the lowest peak memory among the custom models in all three cross-dataset settings. The repeated memory values across datasets reflect the fixed profiling setup, including batch size, context length, numerical precision, and hardware.

C.4 Pretrained 125M Models

Figure 5 compares Q-Interference with GPT-Neo-125M and OPT-125M. These two models are pre-trained decoder-only language models, while Q-Interference is trained from scratch in our controlled experimental setup. Therefore, this comparison should be read as contextual evidence rather than as a parameter-matched baseline comparison. The pretrained models achieve stronger final test perplexity, which is expected because they benefit from large-scale pretraining. However, Q-Interference remains competitive in training memory and uses lower peak GPU memory than both pretrained references on TinyStories, pile-10k, and small-C4. This result suggests that the proposed phase-aware attention design can be implemented practically, even when compared against strong pretrained 125M-scale reference models.

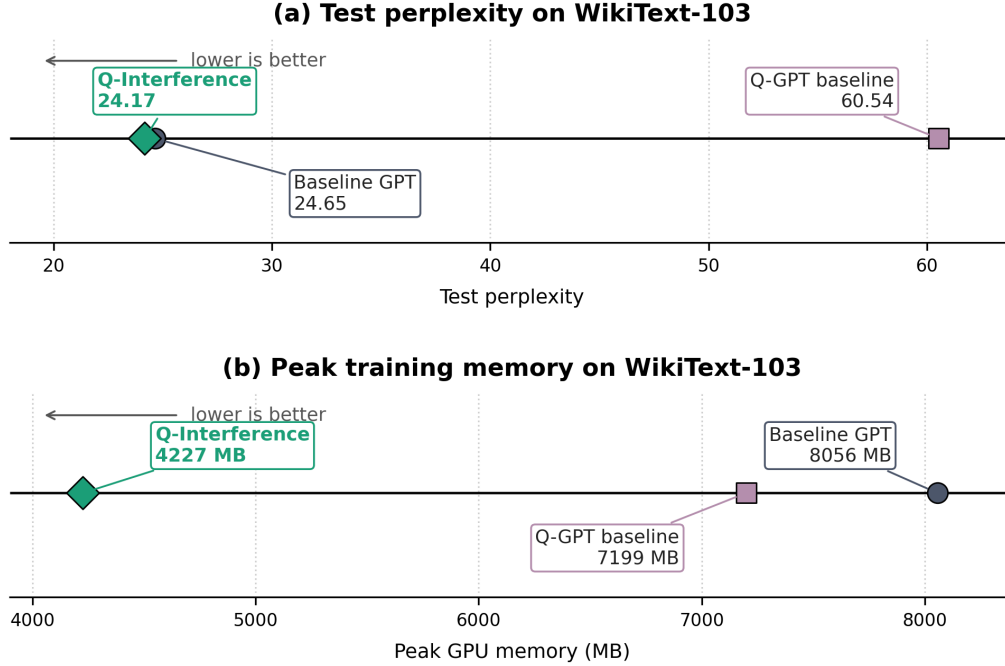


Figure 3: Controlled comparison on the WikiText-103 dataset. The top ruler reports test perplexity and the bottom ruler reports peak training GPU memory. Each marker represents one internal model, and each label reports the exact value. Q-Interference achieves the best internal test perplexity and the lowest peak GPU memory in this controlled setting.

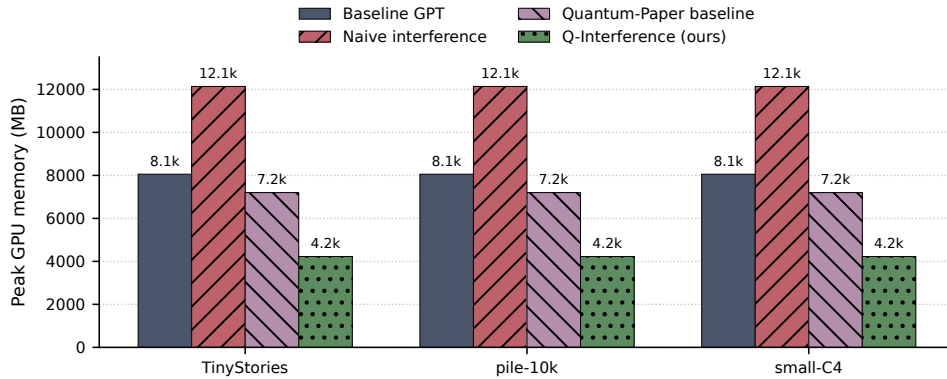


Figure 4: Peak GPU memory across the custom model family. Q-Interference consistently uses less memory than the standard GPT baseline, naive interference, and the Q-GPT baseline under the reported setting. The naive interference model highlights the cost of materializing the phase-aware interaction tensor.

D Limitations and Discussion

The results support Q-Interference as a practical and well-motivated phase-aware attention design. The strongest evidence in this work is that the proposed exact reformulation makes richer interference-based attention trainable in a standard GPT pipeline while substantially reducing the additional memory cost of naive phase-aware computation. The ablations further show that the phase term contributes a small but consistent modeling benefit on top of this practical design.

Dataset	Test perplexity rank lower is better			Peak memory rank lower is better		
WikiText-103	1 Neo 19.44	2 OPT 20.43	3 Q-Int 24.17	1 Neo 4.1k	2 OPT 4.2k	3 Q-Int 4.2k
TinyStories	1 OPT 4.14	2 Neo 4.57	3 Q-Int 5.69	1 Q-Int 4.2k	2 OPT 6.2k	3 Neo 9.9k
pile-10k	1 Neo 10.91	2 OPT 16.24	3 Q-Int 116.93	1 Q-Int 4.2k	2 OPT 6.2k	3 Neo 9.9k
small-C4	1 OPT 32.44	2 Neo 34.65	3 Q-Int 246.90	1 Q-Int 4.2k	2 OPT 6.2k	3 Neo 9.9k

1 GPT-Neo-125M
 2 OPT-125M
 3 Q-Interference

Ranks are computed separately within each dataset and metric. External models are contextual references.

Figure 5: Pretrained 125M model comparison. GPT-Neo-125M and OPT-125M are pretrained decoder-only models, while Q-Interference is trained from scratch in the controlled setup used in this work. The pretrained references obtain stronger final language-modeling quality, but Q-Interference shows a favorable memory profile and uses lower peak GPU memory on three of the four datasets. The comparison is intended to contextualize Q-Interference rather than to serve as a parameter-matched baseline.

At the same time, the method should be interpreted with appropriate scope. Q-Interference is not intended as a full replacement for all strong pretrained GPT transformer baselines, and the current study focuses on controlled GPT-style language modeling rather than large-scale pretraining or downstream transfer. Even so, the results suggest that phase-aware quantum-inspired interactions are worth studying as a richer attention family, especially when paired with an exact memory-efficient reformulation that makes them practical.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The main claims are stated in the abstract and Introduction.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Limitations are discussed in the Appendix D at page 17.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: The assumptions and derivations are provided in Appendix A. The appendix proves the phase-aware score, exact factorization, memory accounting and compatibility with causal GPT attention.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The current submission describes the experimental setup.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All dataset are public. We have provided our project git link in the abstract.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: The paper reports the datasets, model families, context length, hardware, precision, and evaluation metrics in Section 3. However, it does not explicitly specify all training details such as optimizer type, learning rate, batch size, number of epochs, seed choice, and hyperparameter selection procedure.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper reports single-run values for validation loss, test loss, test perplexity, and peak GPU memory, but it does not report error bars, confidence intervals, statistical significance tests, or multi-seed mean $\pm$ standard deviation for the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We provide all key information in our submission.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Yes, we maintain all the ethics related to code.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: There is no ethical consideration right now.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.

- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper does not contain such resources.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Yes, we have cited properly.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: There is no new asset.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing is in here.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper doesn't include human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: We used LLM for editing, grammar checking and word choice.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.